AI PORTFOLIO PROJECT SUMMARY

1. Project Overview

This project is a production-ready AI RAG (Retrieval-Augmented Generation) Chat Widget integrated into a personal portfolio website. It acts as an intelligent virtual agent that can answer recruiter or user queries in real-time based on the owner's CV and technical blog posts, leveraging modern AI infrastructures.

2. System Architecture

The system is built as a separated Client-Server model (Monorepo architecture structure):

- **Front-End (Client):** Next.js (App Router) with React and TypeScript, styled with Tailwind CSS. It handles the interactive user interface, message state management, and real-time streaming data parsing. Deployed globally on Vercel.
- **Back-End (Server):** Powered by Python FastAPI. It coordinates the RAG logic and exposes secure API endpoints. Document ingestion runs separately through the `ingest.py` script. Deployed on Render.
- **Vector Store & AI Model:** Pinecone (managed vector database) stores the document chunks and embeds them server-side with `llama-text-embed-v2` (1024 dimensions, cosine similarity). Google Gemini synthesizes context-aware responses (model set by `GEMINI_CHAT_MODEL`, default `gemini-3.5-flash-lite`).
- **Cache:** A managed Redis instance (Redis Cloud) stores answers to identical questions for 24 hours.

3. Core Technical Concepts

- **RAG (Retrieval-Augmented Generation):** Dynamically injects relevant document snippets directly into the LLM context loop to prevent hallucinations and provide accurate, source-grounded answers.
- **Data Flow Pipeline:** User Query → API Request → Redis cache lookup (hit: answer returned immediately, `X-Cache: HIT`) → on a miss, Pinecone semantic search (top 4 chunks, query embedded by Pinecone) → Prompt Synthesis → Gemini Response (streamed) → Front-End Stream Output. The finished answer is saved to Redis only after the stream completes.
- **Data Ingestion:** `ingest.py` reads the PDF and TXT files in `backend/data/`, splits them into 500-character chunks (50 overlap) and upserts the raw text to Pinecone. Chunk IDs are deterministic, so re-running overwrites instead of duplicating. Run it once after creating the index and again whenever the data changes.
- **Resilience:** Redis errors and timeouts (2 s) are treated as a cache miss, so chat keeps working without the cache. Gemini and Pinecone failures return a 502 with a generic message, and partial answers are never cached.
- **Production Security:** CORS configuration restricts untrusted external requests while exposing key endpoints securely via environment variables (.env masking). API keys and connection strings stay on the server.
- **Markdown & Syntax Highlight:** react-markdown and prismjs render the assistant's answers (lists, tables, highlighted code) in the chat widget, loaded lazily the first time the chat is opened.

4. Production Deployment Checklist

Component	Platform	Build Command	Start Command / Root
Front-End	Vercel	`npm run build`	Root: frontend/
Back-End	Render	`pip install -r requirements.txt`	uvicorn main:app --host 0.0.0.0 --port $PORT Root: backend/

Where	Environment variables
Back-End (Render)	GOOGLE_API_KEY, PINECONE_API_KEY, PINECONE_INDEX_NAME, REDIS_URL (optional: without it the server runs with no cache), GEMINI_CHAT_MODEL (optional), PINECONE_NAMESPACE (optional)
Front-End (Vercel)	NEXT_PUBLIC_API_URL (public backend address). It is inlined at build time, so redeploy after changing it.

One-time setup: create the Pinecone index (a `chunk_text` field with integrated embedding plus a `title` field), then run `python ingest.py` from `backend/`. Pinecone indexes asynchronously, so allow about a minute before testing.

5. Scalability & Advanced Enhancements

Delivered:

- **Pinecone Cloud:** Migrated from FAISS to Pinecone to get a persistent cloud index and avoid the cold-start re-indexing overhead caused by Render's ephemeral storage container life-cycles.
- **Redis Layer:** A Redis caching layer intercepts identical queries, dropping LLM token usage to 0 and reducing response latency to milliseconds. Cache keys are normalized and tied to the model, prompt and data content, so edits invalidate old answers automatically.
- **Markdown & Syntax Highlight:** react-markdown and prismjs give assistant answers IDE-grade formatting for code snippets.

Next steps:

- Rate limiting per IP (for example slowapi, storing counters in the existing Redis) plus provider-side quota caps.
- Semantic cache, so differently worded versions of the same question share one answer.
- Automated ingestion in CI and cleanup of stale chunks left in Pinecone after content edits.
- Automated tests with mocked LLM, Pinecone and Redis, and metrics such as cache hit ratio and time to first token.